Aula 7
Teste Estrutural (Caixa Branca) e Teste em Ambiente Web

Herysson R. Figueiredo
herysson.figueiredo@ufn.edu.br

Base da aula: Pressman & Maxim (2021), caps. 19 e 21; Martins (2016), cap. 2.

Objetivos da aula

- Compreender o teste caixa branca (estrutural) e sua relação com a estrutura interna do código.
- Representar o fluxo de controle de um programa e identificar caminhos independentes.
- Calcular e interpretar a complexidade ciclomática e discutir critérios de cobertura.
- Distinguir critérios baseados em fluxo de controle, fluxo de dados e ciclos.
- Reconhecer os principais focos de teste em aplicações Web.

Fonte: Pressman & Maxim (2021, Seções 19.4 e 21.4–21.8); Martins (2016, Seções 2.5 e 2.7).

Mapa da aula

01

Teste estrutural

Código → grafo → caminhos → cobertura

02

Teste em ambiente Web

Conteúdo → interface → navegação → riscos não funcionais

Organização didática elaborada a partir de Pressman & Maxim (2021) e Martins (2016).

Teste Estrutural (Caixa Branca)

O que é teste caixa branca?

Foco

- Estrutura de controle e lógica interna.
- Código-fonte como principal artefato para derivar casos de teste.
- Comportamento das estruturas internas do programa.

Objetivo

- Exercitar partes relevantes da implementação.
- Projetar casos que revelem defeitos ligados à lógica e ao fluxo.
- Complementar os testes funcionais.

Fonte: Pressman & Maxim (2021, Seção 19.4); Martins (2016, pp. 46–47, Seção 2.5).

O que precisamos para aplicar a técnica?

- Acesso ao código-fonte ou a uma representação suficientemente detalhada da estrutura interna.
- Conhecimento sobre decisões, repetições, estruturas de dados e possíveis fluxos de execução.
- Casos de teste definidos a partir da implementação, sem dispensar a especificação e os resultados esperados.
- Um critério de cobertura para orientar quando o conjunto de testes é considerado suficiente para o objetivo escolhido.

Fonte: Martins (2016, pp. 46–48); Pressman & Maxim (2021, Seção 19.4).

O que os casos estruturais devem exercitar?

- Caminhos independentes do módulo.
- Decisões lógicas nos resultados verdadeiro e falso.
- Ciclos em seus limites e dentro de suas fronteiras operacionais.
- Estruturas de dados internas, buscando verificar sua validade.

Fonte: Pressman & Maxim (2021, Seção 19.4).

Grafo de Fluxo de Controle (GFC)

Representação do fluxo de controle usada para visualizar a sequência de execução e seus desvios.

Nó	bloco de comandos executados de forma sequencial
Aresta	transferência de controle entre nós
Região	área delimitada por nós e arestas; a região externa também é contada

Fonte: Pressman & Maxim (2021, Seção 19.4.1); Martins (2016, p. 48).

Exemplo: do código para o fluxo de controle

```java
double calcularFrete(double valor, boolean premium) {
    double frete = 20.0;
    if (valor >= 200) {
        frete = 0.0;
    } else if (premium) {
        frete = 10.0;
    }
    return frete;
}
```

- Há duas decisões: valor >= 200 e premium.
- Cada decisão cria alternativas de fluxo.
- O grafo permitirá derivar caminhos e casos de teste.

Exemplo elaborado para a aula com base nos conceitos de Pressman & Maxim (2021, Seção 19.4.1).

Grafo do exemplo

```
double calcularFrete(double valor, boolean premium) {
    double frete = 20.0;
    if (valor >= 200) {
        frete = 0.0;
    } else if (premium) {
        frete = 10.0;
    }
    return frete;
}
```

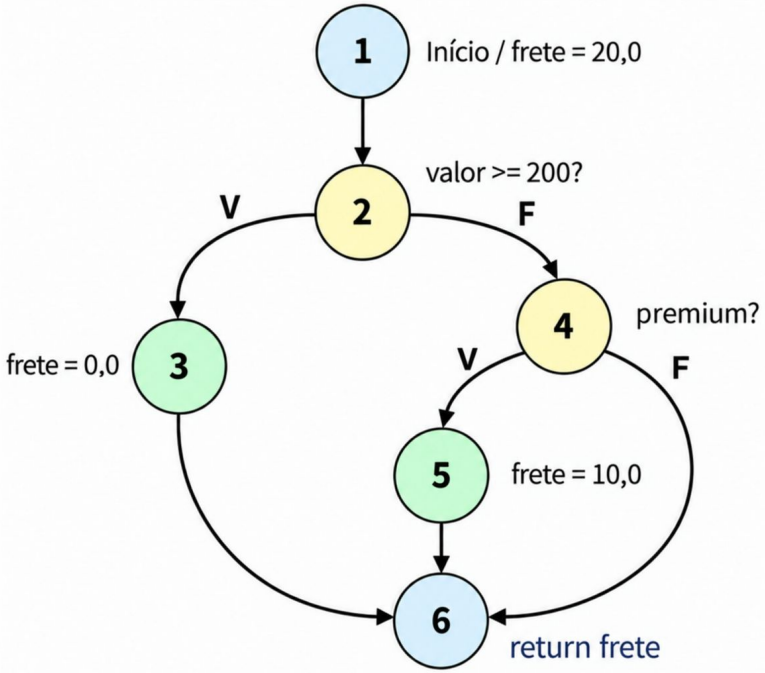


Figura de elaboração própria. Conceitos de grafo de fluxo: Pressman & Maxim (2021, Seção 19.4.1); Martins (2016, pp. 48–49).

Notação Grafo de Fluxo

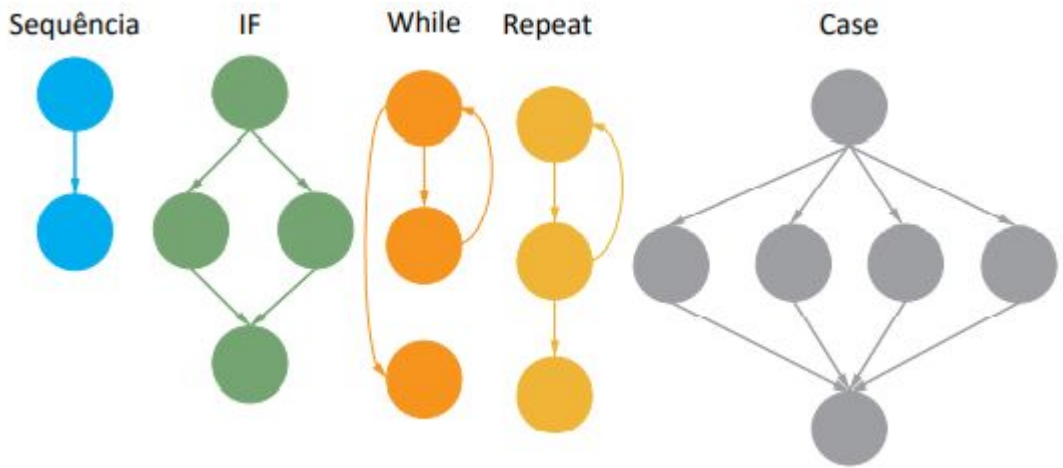


Figura de elaboração própria. Conceitos de grafo de fluxo: Pressman & Maxim (2021, Seção 19.4.1); Martins (2016, pp. 48–49).

Exemplo 2: do código para o fluxo de controle

```
1:  public int processarPedidos(List<Pedido> pedidos) {
2:      int aprovados = 0;
3:
4:      for (Pedido pedido : pedidos) {
5:          if (pedido.getValor() >= 500) {
6:              registrarPedido(pedido);
7:              aprovados++;
8:          } else if (pedido.isClienteVip()) {
9:              marcarParaRevisao(pedido);
10:         } else {
11:             arquivarPedido(pedido);
12:         }
13:     }

        return aprovados;
    }
```

A partir do código a seguir, modele o grafo de fluxo de controle, identificando os nós e as arestas do programa.

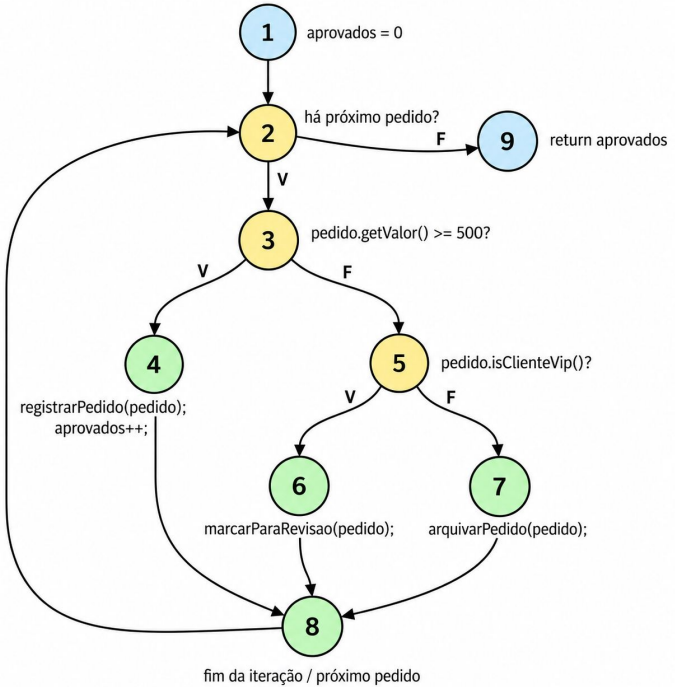
https://app.diagrams.net/ - draw.io

Exemplo elaborado para a aula com base nos conceitos de Pressman & Maxim (2021, Seção 19.4.1).

Exemplo 2: do código para o fluxo de controle

```
1:  public int processarPedidos(List<Pedido> pedidos) {
2:      int aprovados = 0;
3:
4:      for (Pedido pedido : pedidos) {
5:          if (pedido.getValor() >= 500) {
6:              registrarPedido(pedido);
7:              aprovados++;
8:          } else if (pedido.isClienteVip()) {
9:              marcarParaRevisao(pedido);
10:         } else {
11:             arquivarPedido(pedido);
12:         }
13:     }

        return aprovados;
    }
```



Exemplo elaborado para a aula com base nos conceitos de Pressman & Maxim (2021, Seção 19.4.1).

Caminho independente

• É um caminho que introduz pelo menos uma nova aresta — ou uma nova condição — em relação aos caminhos já considerados.

• Um conjunto-base é formado por caminhos linearmente independentes.

• Executar o conjunto-base é a ideia central do teste de caminho básico.

• O conjunto-base não é único: podem existir diferentes conjuntos válidos para o mesmo grafo.

Fonte: Pressman & Maxim (2021, Seção 19.4.1).

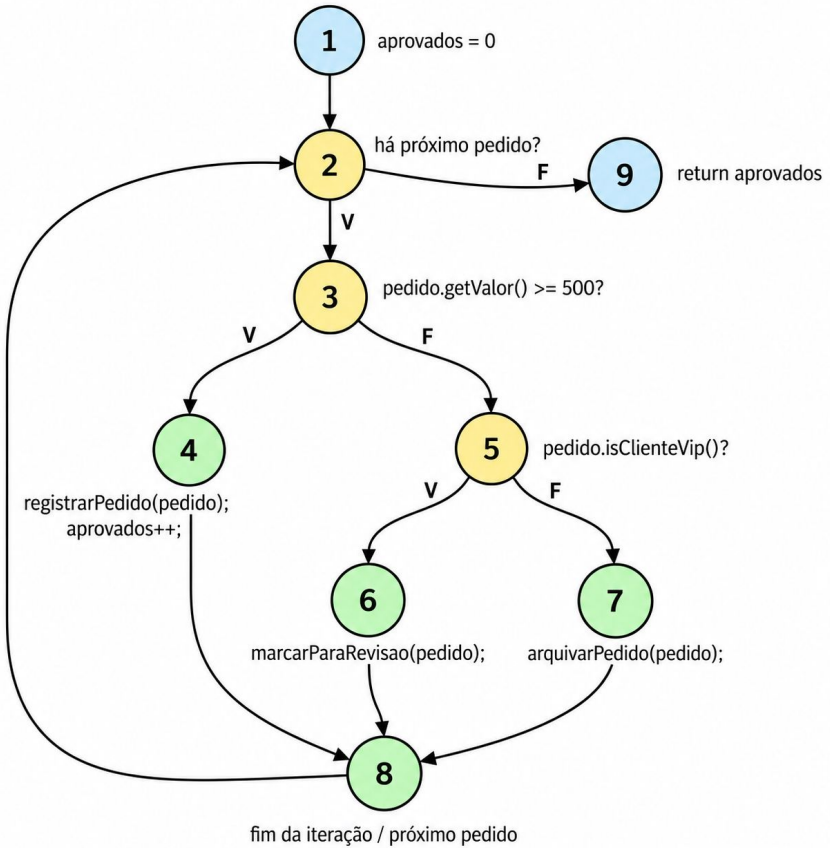
Caminho independente

Caminhos independentes:

1. 1-2-9
2. 1-2-3-4-8-2-9
3. 1-2-3-5-6-8-2-9
4. 1-2-3-5-7-8-2-9

O caminho:

1-2-3-4-8-2-3-5-6-8-2-9 **NÃO** é independente.



Fonte: Pressman & Maxim (2021, Seção 19.4.1).

Complexidade ciclomática

O que representa

- Métrica baseada no grafo de fluxo.
- Relaciona-se à complexidade lógica do programa.
- Fornece o número de caminhos independentes do conjunto-base.

Como usar no teste

- Ajuda a dimensionar o conjunto-base de caminhos.
- Orienta o projeto de casos para o teste de caminho básico.
- Valores maiores indicam estrutura de controle mais complexa.

Fonte: Pressman & Maxim (2021, Seção 19.4.1); McCabe (1976).

Complexidade ciclomática — formas de cálculo

Regiões

$$V(G) = R$$

R = número de regiões do grafo

Arestas e nós

$$V(G) = E - N + 2$$

E = arestas; N = nós

Nós-predicado

$$V(G) = P + 1$$

P = nós de decisão/predicado

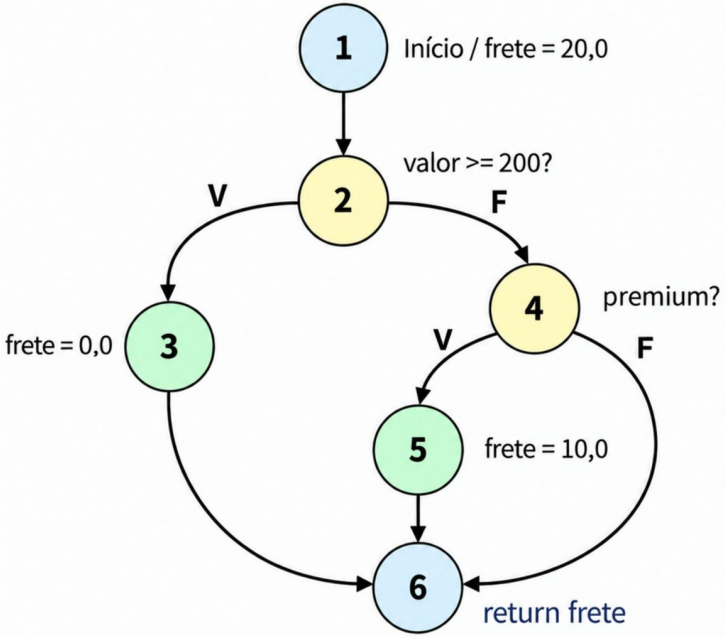
Fonte: Pressman & Maxim (2021, Seção 19.4.1). Para o grafo conectado apresentado nesta aula.

Regiões

Regiões

V(G) = R

R = número de regiões do grafo

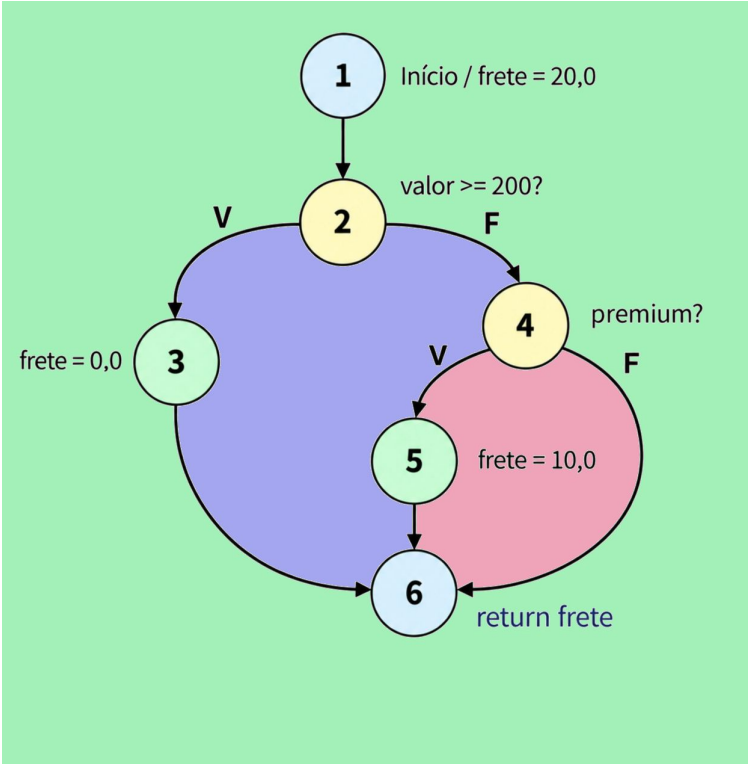


Regiões

Regiões

V(G) = R

R = número de regiões do grafo



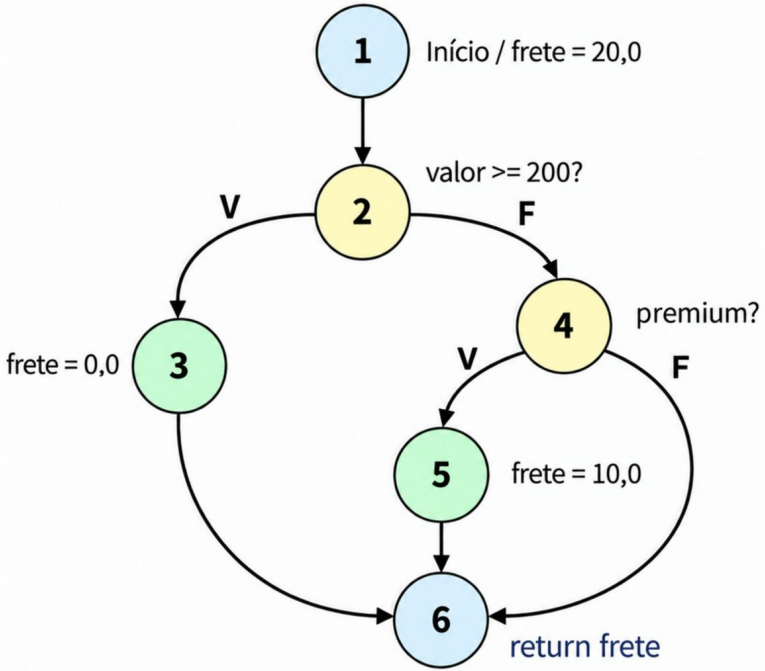
Arestas e Nós

Arestas e nós

$$V(G) = E - N + 2$$

E = arestas; N = nós

- 6 nós (N = 6)
- 7 arestas (E = 7)



Fórmulas: Pressman & Maxim (2021, Seção 19.4.1).

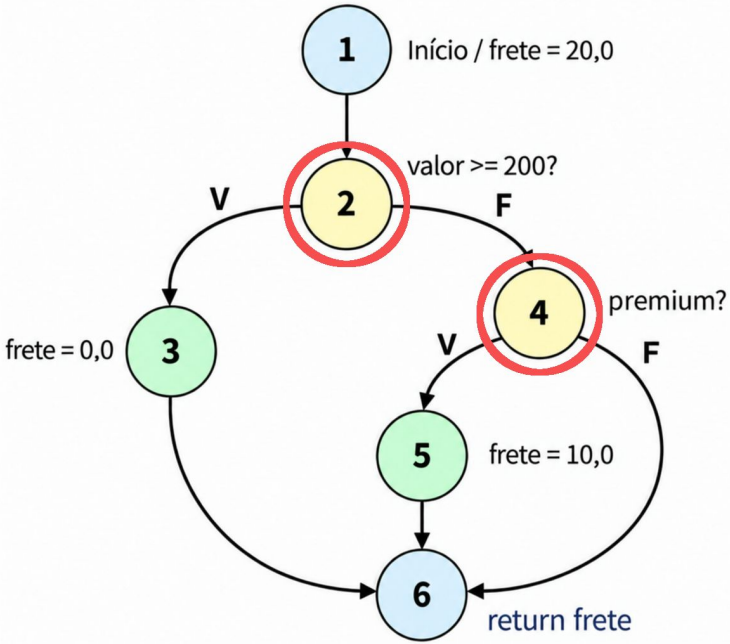
Nós-predicado

Nós-predicado

V(G) = P + 1

P = nós de decisão/predicado

- Possui 2 ou mais arestas de saída.



Fonte: Pressman & Maxim (2021, Seção 19.4.1). Para o grafo conectado apresentado nesta aula.

Teste de caminho básico — sequência de trabalho

1. Construir o grafo de fluxo
2. Calcular V(G)
3. Determinar um conjunto-base
4. Projetar casos que executem cada caminho

Síntese didática baseada em Pressman & Maxim (2021, Seção 19.4.1).

Grafo do exemplo

```
double calcularFrete(double valor, boolean premium) {
    double frete = 20.0;
    if (valor >= 200) {
        frete = 0.0;
    } else if (premium) {
        frete = 10.0;
    }
    return frete;
}
```

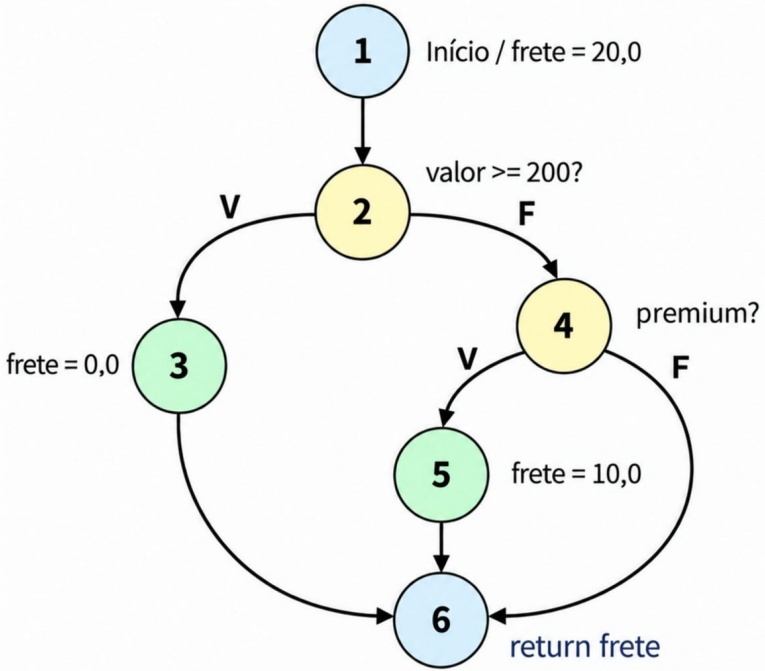


Figura de elaboração própria. Conceitos de grafo de fluxo: Pressman & Maxim (2021, Seção 19.4.1); Martins (2016, pp. 48–49).

Conjunto-base do exemplo

P1	**1 → 2(T) → 4 → 6**	valor = 250; premium = false
P2	**1 → 2(F) → 3(T) → 5 → 6**	valor = 100; premium = true
P3	**1 → 2(F) → 3(F) → 6**	valor = 100; premium = false

Como V(G)=3, buscamos três caminhos independentes no conjunto-base.

Exemplo elaborado para a aula com base em Pressman & Maxim (2021, Seção 19.4.1).

Cobertura de ramos / todas as arestas

Critério

- As alternativas verdadeira e falsa de cada decisão devem ser exercitadas.
- No grafo, busca-se percorrer todas as arestas relevantes.
- Pode exigir mais casos que a cobertura de comandos.

Interpretação

- A cobertura aumenta o rigor sobre a lógica de decisão.
- Mesmo 100% de ramos não garante que todos os defeitos tenham sido revelados.
- Critérios mais exigentes tendem a aumentar o esforço de teste.

Fonte: Martins (2016, pp. 50–51).

Cobertura de ramos / todas as are

CT1: valor = 250, premium = false
→ 1 → 2 → 3 → 6

CT2: valor = 100, premium = true
→ 1 → 2 → 4 → 5 → 6

CT3: valor = 100, premium = false
→ 1 → 2 → 4 → 6

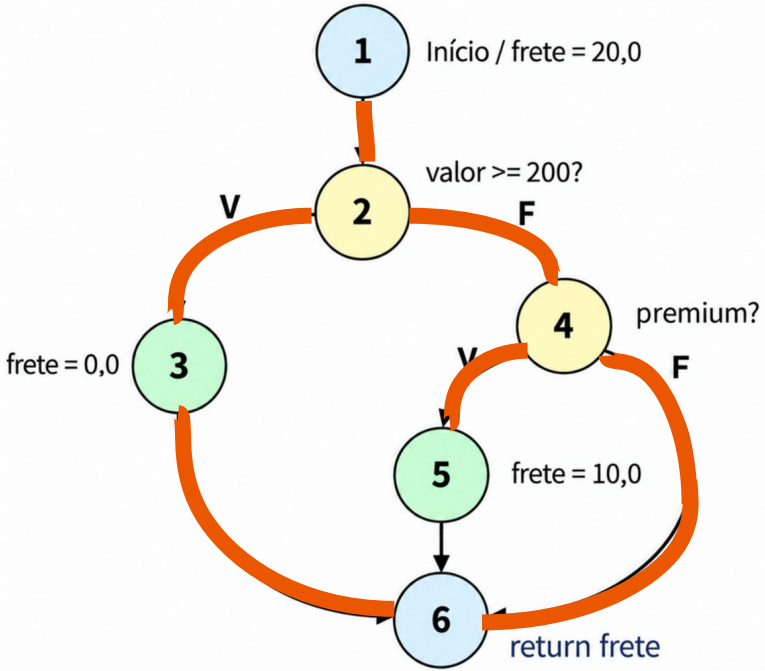


Figura de elaboração própria. Conceitos de grafo de fluxo: Pressman & Maxim (2021, Seção 19.4.1); Martins (2016, pp. 48–49).

Cobertura de comandos / todos os nós

Critério

- Cada comando executável deve ser exercitado ao menos uma vez.
- No grafo, busca-se executar todos os nós.
- É um critério de fluxo de controle.

Interpretação

- 100% significa que os comandos-alvo foram executados.
- Não implica que todas as decisões ou combinações tenham sido exercitadas.
- Serve como medida de cobertura, não como prova de ausência de defeitos.

Fonte: Martins (2016, pp. 49–50).

Cobertura de comandos / todos os nós

CT1: valor = 250, premium = false
→ 1 → 2 → 3 → 6

CT2: valor = 100, premium = true
→ 1 → 2 → 4 → 5 → 6

Nós percorridos: {1,2,3,4,5,6}

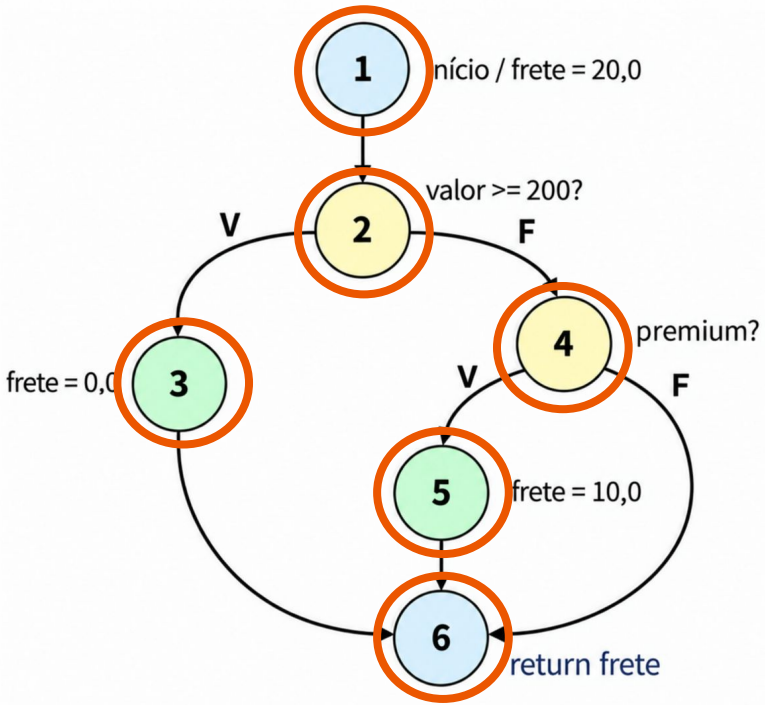


Figura de elaboração própria. Conceitos de grafo de fluxo: Pressman & Maxim (2021, Seção 19.4.1); Martins (2016, pp. 48–49).

Cobertura de comandos ≠ cobertura de ramos

```java
double calcularFrete(double valor, boolean premium) {
    double frete = 20.0;
    if (valor >= 200) {
        frete = 0.0;
    } else if (premium) {
        frete = 10.0;
    }
    return frete;
}
```

Conjunto de testes	Comandos	Ramos
250 / false + 100 / true	100%	Não
250 / false + 100 / true + 100 / false	100%	100%

O terceiro caso não acrescenta novos comandos, mas exercita o ramo premium = false.

Exemplo elaborado para a aula a partir dos critérios apresentados por Martins (2016, pp. 49–51).

Teste de condição

- Amplia a atenção sobre as expressões lógicas usadas nas decisões.
- É útil quando um predicado contém várias condições combinadas por operadores lógicos.
- O objetivo é projetar dados que exercitem resultados relevantes das condições e revelem defeitos na lógica.
- Critérios de condição complementam a simples cobertura de ramos.

Fonte: Pressman & Maxim (2021, Seção 19.4.2); Martins (2016, p. 52).

Teste baseado em fluxo de dados

- Seleciona caminhos de acordo com a localização das definições e dos usos de variáveis.
- Procura exercitar relações entre a atribuição de um valor e os pontos em que esse valor é utilizado.
- Ajuda a revelar problemas que podem não aparecer quando observamos somente nós e arestas.
- É uma técnica de estrutura de controle discutida no contexto do teste caixa branca.

Fonte: Pressman & Maxim (2021, Seção 19.4.2); Martins (2016, pp. 52–53).

Ocorrências de variáveis: def, c-use e p-use

DEF	definição: um valor é atribuído à variável
C-USE	uso computacional: variável usada em expressão ou saída
P-USE	uso predicativo: variável usada em uma decisão que afeta o fluxo

Fonte: Martins (2016, p. 52).

Critério “Todos os Usos”

- Identificar as ocorrências das variáveis relevantes.
- Classificar cada ocorrência como definição, uso computacional ou uso predicativo.
- Relacionar definições e usos (pares d-u).
- Projetar casos de teste que exercitem os usos definidos pelo critério.

Fonte: Martins (2016, p. 53).

Teste baseado em fluxo de dados

```
double calcularFrete(double valor, boolean premium) {
    double frete = 20.0;          // def

    if (valor >= 200) {           // p-use de valor
        frete = 0.0;              // def
    } else if (premium) {         // p-use de premium
        frete = 10.0;             // def
    }

    return frete;                 // c-use
}
```

Onde frete **recebe** valores?

Onde esse valor é efetivamente **usado**?

Fonte: Martins (2016, p. 52).

Teste baseado em fluxo de dados

```
double calcularFrete(double valor, boolean premium) {
    double frete = 20.0;          // def

    if (valor >= 200) {           // p-use de valor
        frete = 0.0;              // def
    } else if (premium) {         // p-use de premium
        frete = 10.0;             // def
    }

    return frete;                 // c-use
}
```

DEF frete = 20.0; frete = 0.0; frete = 10.0;

C-USE valor >= 200 premium

P-USE return frete;

Fonte: Martins (2016, p. 52).

Teste baseado em fluxo de dados

```
double calcularFrete(double valor, boolean premium) {
    double frete = 20.0;          // def

    if (valor >= 200) {           // p-use de valor
        frete = 0.0;              // def
    } else if (premium) {         // p-use de premium
        frete = 10.0;             // def
    }

    return frete;                 // c-use
}
```

Quantas definições diferentes da variável frete podem alcançar o return?

Fonte: Martins (2016, p. 52).

Teste baseado em fluxo de dados

```
double calcularFrete(double valor, boolean premium) {
    double frete = 20.0;          // def

    if (valor >= 200) {           // p-use de valor
        frete = 0.0;              // def
    } else if (premium) {         // p-use de premium
        frete = 10.0;             // def
    }

    return frete;                 // c-use
}
```

Caso de teste	Entrada	Definição exercitada	Resultado
CT1	valor = 100, premium = false	frete = 20.0	20.0
CT2	valor = 250, premium = qualquer	frete = 0.0	0.0
CT3	valor = 100, premium = true	frete = 10.0	10.0

Fonte: Martins (2016, p. 52).

Teste de ciclos (loops)

- Concentra-se exclusivamente na validade das construções de ciclo.
- Pressman distingue ciclos simples e ciclos aninhados.
- Os casos devem explorar fronteiras de repetição, pois defeitos são frequentes nesses limites.
- O objetivo não é testar um número arbitrário de iterações, mas selecionar iterações representativas e limites.

Fonte: Pressman & Maxim (2021, Seção 19.4.2).

Ciclo simples: conjunto de testes sugerido

Considere n como o número máximo permitido de passagens pelo ciclo.

0	pular o ciclo
1	uma passagem
2	duas passagens
m	m passagens, com m < n
n−1, n, n+1	próximo ao limite

Fonte: Pressman & Maxim (2021, Seção 19.4.2).

Ciclos aninhados: estratégia

- Começar pelo ciclo mais interno, mantendo os ciclos externos em valores mínimos.
- Aplicar os testes de ciclo simples ao ciclo interno.
- Avançar para fora, testando o próximo ciclo e mantendo os demais em valores típicos ou mínimos.
- Continuar até que todos os níveis de aninhamento relevantes tenham sido exercitados.

Fonte: Pressman & Maxim (2021, Seção 19.4.2).

O que a cobertura nos diz — e o que não diz

Ela ajuda a

- Medir quais estruturas foram exercitadas.
- Identificar lacunas no conjunto de testes.
- Comparar o conjunto executado com um critério explícito.

Ela não prova que

- Todos os defeitos foram encontrados.
- O software satisfaz todos os requisitos.
- Outros dados ou sequências não revelarão novos problemas.

Fonte: Martins (2016, p. 51); Pressman & Maxim (2021, Seções 19.4–19.4.2).

Atividade 1 — Teste estrutural

```
int classificar(int n) {
    int r = 0;
    if (n < 0) {
        r = -1;
    } else if (n % 2 == 0) {
        r = 2;
    } else {
        r = 1;
    }
    return r;
}
```

- Desenhe o grafo de fluxo de controle.
- Calcule V(G).
- Defina um conjunto-base de caminhos.
- Proponha casos de teste para cobrir todos os ramos.

Atividade 1 — discussão / resposta esperada

Duas decisões → V(G) = P + 1 = 3.

Caminho	Exemplo de entrada	Saída
n < 0	−5	−1
n ≥ 0 e par	4	2
n ≥ 0 e ímpar	5	1

Os três casos exercitam os resultados possíveis das duas decisões e formam um conjunto-base natural para este exemplo.

Resposta elaborada para a aula a partir do método de caminho básico de Pressman & Maxim (2021).

Atividade 2 — Teste estrutural

```java
public static int buscaBinaria(int[] array, int valor) {
    int inicio = 0;
    int fim = array.length - 1;
    int retorno = -1;

    while (inicio <= fim) {
        int meio = (inicio + fim) / 2;

        if (array[meio] == valor) {
            retorno = meio;
            fim = -1;
        } else if (valor > array[meio]) {
            inicio = meio + 1;
        } else {
            fim = meio - 1;
    }

    return retorno;
}
```

- Desenhe o grafo de fluxo de controle.
- Calcule V(G).
- Defina um conjunto-base de caminhos.
- Proponha casos de teste para cobrir todos os ramos.

Atividade 3 — Teste estrutural

```java
public static void bubbleSort(int[] array) {
    int n = array.length;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (array[j] > array[j + 1]) {
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
            }
        }
    }
}
```

- Desenhe o grafo de fluxo de controle.
- Calcule V(G).
- Defina um conjunto-base de caminhos.
- Proponha casos de teste para cobrir todos os ramos.

Parte II — Teste em Ambiente Web

Por que o contexto Web merece atenção específica?

• Aplicações Web operam em rede e interoperam com navegadores, servidores, bancos de dados e serviços.

• Conteúdo, interface e navegação são parte essencial da experiência do usuário.

• Variações de configuração podem alterar o comportamento observado.

• Segurança e desempenho tornam-se riscos centrais quando muitos usuários e componentes interagem.

Fonte: Pressman & Maxim (2021, Seções 21.4–21.5); Martins (2016, pp. 58–60).

Uma sequência para testar uma WebApp

1. Revisar conteúdo
2. Revisar modelo de interface
3. Revisar modelo de navegação
4. Testar interação da interface
5. Testar componentes funcionais
6. Testar caminhos de navegação
7. Testar configurações
8. Testar segurança
9. Testar desempenho
10. Testar com usuários finais

Síntese dos passos apresentados por Pressman & Maxim (2021, Seção 21.4).

Principais focos de teste em ambiente Web

Conteúdo
correção e consistência

Interface
interação e apresentação

Navegação
caminhos e mecanismos

Componente
funcionalidades e serviços

Configuração
ambientes e compatibilidade

Segurança
vulnerabilidades e proteção

Desempenho
tempo de resposta e carga

Fonte: Martins (2016, Seção 2.7); Pressman & Maxim (2021, Seções 21.4–21.8).

Teste de conteúdo

Objetivo

- Descobrir erros de sintaxe e correção textual.
- Verificar semântica, exatidão e integridade das informações.
- Avaliar a organização e a estrutura do conteúdo apresentado.

Exemplos de problemas

- Informação incorreta ou desatualizada.
- Link associado ao conteúdo está incorreto.
- Imagem, legenda ou referência inconsistentes.
- Conteúdo confuso, incompleto ou inadequadamente organizado.

Fonte: Pressman & Maxim (2021, Seção 21.5.1); Martins (2016, p. 58).

Teste de interface

Objetivo

- Exercitar mecanismos de interação da interface.
- Verificar regras de projeto, estética e conteúdo visual.
- Testar mecanismos no contexto dos casos de uso e em diferentes ambientes.

Exemplos de problemas

- Formulário envia dados de forma incorreta.
- Elemento de interface é ambíguo ou inconsistente.
- Comportamento difere entre navegadores.
- Mensagem ao usuário não explica adequadamente o erro.

Fonte: Pressman & Maxim (2021, Seção 21.5.2); Martins (2016, p. 58).

Teste de navegação

Objetivo

- Garantir que mecanismos de navegação funcionem.
- Exercitar caminhos/jornadas definidos para o usuário.
- Verificar acessibilidade dos nós de navegação e retorno entre páginas/estados.

Perguntas úteis

- Todos os nós esperados podem ser alcançados?
- Links, botões e mecanismos de retorno funcionam?
- O usuário entende onde está na arquitetura do site?
- Caminhos alternativos conduzem ao resultado correto?

Fonte: Pressman & Maxim (2021, Seção 21.5.3); Martins (2016, p. 58).

Teste de componente em aplicações Web

O que pode ser componente

- Página com conteúdo, navegação e processamento.
- Serviço oferecido ao usuário, como relatório ou consulta.
- Componente de infraestrutura, como banco de dados ou serviço Web.

Foco do teste

- Encontrar defeitos funcionais do componente.
- Verificar colaboração e integração com outros componentes.
- Exercitar o componente no contexto da arquitetura prevista.

Fonte: Martins (2016, p. 59); Pressman & Maxim (2021, Seção 21.4).

Teste de configuração

Objetivo

- Identificar defeitos que aparecem em ambientes específicos.
- Avaliar compatibilidade entre plataforma, navegador, hardware, servidor e protocolos.
- Selecionar combinações relevantes para o ambiente real de uso.

Estratégia prática

- Construir uma matriz de configurações.
- Priorizar as combinações mais usadas ou mais críticas.
- Registrar ambiente e versão nos resultados do teste.

Fonte: Martins (2016, p. 59); Pressman & Maxim (2021, Seção 21.4).

Teste de segurança

Objetivo

- Investigar vulnerabilidades em cliente, rede e servidor.
- Verificar se mecanismos de proteção resistem a usos indevidos.
- Descobrir pontos fracos antes que sejam explorados.

Exemplos de superfícies

- Autenticação e autorização.
- Entrada de dados e scripts maliciosos.
- Comunicação cliente–servidor.
- Acesso a dados e serviços protegidos.

Fonte: Pressman & Maxim (2021, Seção 21.7); Martins (2016, p. 59).

Teste de desempenho

O que observar

- Tempo de resposta sob diferentes cargas.
- Comportamento de servidor, rede, banco de dados e demais recursos.
- Componentes responsáveis por degradação de desempenho.

Perguntas de teste

- Como o tempo de resposta varia quando a carga cresce?
- Em que ponto o sistema se torna inaceitável?
- Quais recursos se tornam gargalos?
- O sistema se recupera após picos de uso?

Fonte: Pressman & Maxim (2021, Seção 21.8); Martins (2016, p. 60).

Carga × esforço (stress)

Teste de carga

Avalia o comportamento sob condições de carga definidas e permite observar métricas de desempenho em cenários esperados.

Teste de esforço (stress)

Submete o sistema a condições extremas para observar degradação, falhas e capacidade de recuperação.

Fonte: Pressman & Maxim (2021, Seção 21.8).

Caixa branca também aparece no contexto Web

Aplicações típicas

- Teste unitário de serviços, controladores e funções de negócio.
- Cobertura de ramos e condições no código do servidor.
- Fluxo de dados em componentes que processam entradas e persistência.

Complementaridade

- Interface e navegação são fortemente orientadas ao comportamento externo.
- O mesmo sistema pode combinar testes caixa branca e caixa preta.
- A escolha da técnica depende do objetivo e do artefato sob teste.

Fonte: Martins (2016, p. 60); Pressman & Maxim (2021, Seções 19.4 e 21.4).

Atividade 2 — Estratégia de teste para uma WebApp

Cenário: plataforma Web para inscrição em um evento acadêmico, com cadastro, login, submissão de trabalho, pagamento e emissão de comprovante.

- Escolha quatro tipos de teste entre: conteúdo, interface, navegação, componente, configuração, segurança e desempenho.
- Para cada tipo, descreva um risco e um caso de teste.
- Identifique quais testes seriam predominantemente caixa preta e onde um teste caixa branca poderia ser útil.

Atividade elaborada com base nas categorias de Pressman & Maxim (2021) e Martins (2016).

Atividade 2 — exemplos para discussão

Tipo	Risco	Exemplo de teste
Navegação	usuário não conclui a submissão	percorrer cadastro → submissão → comprovante
Configuração	falha em navegador/versão	executar fluxo crítico em combinações priorizadas
Segurança	acesso indevido a trabalhos	tentar acessar recurso sem autorização
Desempenho	degradação no prazo final	simular pico de usuários e medir resposta

Possível uso de caixa branca: cobrir ramos dos serviços de autorização, regras de submissão e processamento de pagamento.

Exemplos elaborados para a aula; categorias de teste baseadas em Pressman & Maxim (2021) e Martins (2016).

Síntese da aula

1. Caixa branca deriva testes da estrutura interna do software.
2. Grafo, caminhos independentes e V(G) apoiam o teste de caminho básico.
3. Cobertura é um critério de adequação do conjunto de testes, não uma prova de ausência de defeitos.
4. Fluxo de controle, fluxo de dados e ciclos oferecem perspectivas complementares.
5. Em WebApps, conteúdo, interface, navegação, configuração, segurança e desempenho ampliam o escopo do teste.

Síntese baseada em Pressman & Maxim (2021) e Martins (2016).

Referências

MARTINS, Marcos Danilo Chiodi. Testes de software. Rio de Janeiro: SESES, 2016.

MCCABE, Thomas J. A complexity measure. IEEE Transactions on Software Engineering, v. SE-2, n. 4, p. 308–320, 1976.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

Observação: exemplos de código, tabelas e figuras desta apresentação foram elaborados para fins didáticos a partir das obras citadas.

Referências bibliográficas utilizadas na elaboração da aula.